

Bancos de Dados Não-Relacionais

Aula 01: O Paradigma Pós-Relacional & Diretrizes da Disciplina

A presentation by

Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

Agenda do Encontro

Diretrizes & Avaliação

Contexto Histórico & Big Data

Gargalos do Relacional

O Movimento NoSQL & Modelos

MongoDB na Prática & Conclusão

Diretrizes & Avaliação

Visão Geral da Disciplina: ADS16

Estrutura do Curso

- **Carga Horária:** 40 horas no semestre.
- **Total de Encontros:** 20 encontros (2h/aula cada).
- **Horário Regular:** Terças-feiras.
- **Ambiente:** Laboratório de Informática.
- **Metodologia:** Aulas teórico-práticas com programação.

Pilares de Aprendizagem

- **Ambiente Docker:** Orquestração rápida de Redis, MongoDB e Neo4j.
- **Engenharia de Dados:** Alta escala, concorrência e particionamento.
- **Modernização:** Introdução a Bancos Vetoriais para Inteligência Artificial.

Sistema de Avaliação da Disciplina

Fórmula Regimental de Média por Período (N1 e N2):

$$\text{Nota} = (\text{AVT} \times 0,40) + (\text{AVP} \times 0,40) + (\text{Listas} \times 0,20)$$

AVT (40%) – Avaliação Teórica

- Prova individual sem consulta.
- Fundamentos, CAP/PACELC, ACID vs BASE e arquitetura.

AVP (40%) – Avaliação Prática

- **AVP1 (N1):** Desafio prático individual em laboratório.
- **AVP2 (N2):** Projeto prático integrado em **duplas**.

Listas (20%) – Avaliação Contínua

- Exercícios práticos de laboratório e fixação.
- Entrega até a data da avaliação teórica do período.

Avaliação Final (AF)

- Prova final de recuperação ao término do semestre.

Diretriz dos Sábados Letivos

10/10/2026 – Sábado Letivo 01

- **Dia Sem Conteúdo Novo.**
- **Oficina Prática de Ferramentas:** Orquestração de containers com Docker Compose.
- Subida e teste de instâncias de Redis e MongoDB.
- Resolução orientada da Lista Preparatória 1.

14/11/2026 – Sábado Letivo 02

- **Dia Sem Conteúdo Novo.**
- **Laboratório de Fixação:** Simulado e revisão prática integrada (Redis + MongoDB).
- Plantão de dúvidas antes de AVP1 (17/11) e AVT1 (24/11).
- Fechamento da Lista de Exercícios 1.

Bibliografia & Referências Recomendadas

Bibliografia Básica

- **SADALAGE, P. J.; FOWLER, M.** NoSQL Essencial: Um Guia Conciso para o Mundo Emergente da Persistência Poliglota. Novatec, 2014.
- **ELMASRI, R.; NAVATHE, S. B.** Sistemas de Banco de Dados. 7. ed. Pearson, 2018.
- **PANIZ, D.** NoSQL: como armazenar os dados de uma aplicação moderna. Casa do Código, 2016.

Bibliografia Complementar & Artigos

- **FROZZA, A. A.; SCHREINER, G. A.; MELLO, R. S.** Projeto de Bancos de Dados NoSQL. SBC, Cap. 2.
- **SHAPIRA, G. et al.** Kafka: The Definitive Guide - Real-Time Data and Stream Processing at Scale. 2. ed. O'Reilly, 2021.
- **GUERRA, J. L.** NoSQL: uma abordagem prática. Novatec, 2021.

Contexto Histórico & Big Data

O Domínio do Modelo Relacional (1970 – 2000)

A Era Codd e a Álgebra Relacional

- Proposto por Edgar F. Codd em 1970 (IBM).
- Base matemática rigorosa na álgebra relacional e cálculo de tuplas.
- Tabelas rígidas com chaves primárias, estrangeiras e integridade referencial.
- **Premissa Arquitetural:** Projetado para operar em um único servidor centralizado.

Garantias Transacionais ACID

- **Atomicidade:** Execução de tudo ou nada na transação.
- **Consistência:** Regras de integridade sempre preservadas.
- **Isolamento:** Transações simultâneas sem interferência.
- **Durabilidade:** Persistência garantida após commit.
- **Hegemonia:** Padrão dominante da indústria por mais de 30 anos (Elmasri & Navathe, 2018).

O Ponto de Inflexão: A Web 2.0 e os 3 Vs

Volume & Velocidade

- **Volume:** Salto de gigabytes para terabytes e petabytes diários gerados por bilhões de usuários simultâneos.
- O volume total excede a capacidade de armazenamento de uma única máquina.
- **Velocidade:** Milhões de escritas/leituras por segundo.
- Exigência de tempo de resposta em escala de milissegundos.

Variedade & Flexibilidade

- Dados heterogêneos: documentos JSON, dados polimórficos, logs, grafos e telemetria IoT.
- Esquemas que evoluem rapidamente (schemaless).
- Impossibilidade de paradas programadas para alterações de esquema (`ALTER TABLE`) sob tráfego global ininterrupto.

Streaming & Ingestão em Tempo Real (Kafka)

O Desafio dos Fluxos Contínuos

- **Shapira et al. (2021) – Kafka The Definitive Guide:** A alta velocidade de dados na Web moderna inviabiliza conexões diretas síncronas com o banco de dados.
- Dispositivos IoT, cliques de usuários e sensores geram tráfego contínuo em rajadas.
- Gravações síncronas bloqueantes sobrecarregam pools de conexões e travam o banco principal.

O Papel do Broker Distribuído

- **Apache Kafka:** Plataforma de mensageria distribuída e processamento de streams.
- **Buffer Elástico:** Atua como intermediário tolerante a falhas que absorve milhões de mensagens/segundo.
- **Desacoplamento:** Consumidores assíncronos leem os tópicos e gravam no banco de destino no seu próprio ritmo.

Arquitetura: Do Streaming à Persistência Poliglota



Takeaway: O streaming desacopla a ingestão massiva e distribui os dados para o banco NoSQL ideal para cada propósito.

As Raízes Históricas do NoSQL

De Carlos Strozzi a Bigtable & Dynamo

- **1998 – Carlos Strozzi:** Usou o termo NoSQL para designar um banco de dados relacional leve sem interface SQL (Frezza et al.).
- **2006 – Google Bigtable:** Artigo seminal de Chang et al. sobre armazenamento distribuído esparsa e estruturado.
- **2007 – Amazon Dynamo:** Artigo seminal de DeCandia et al. propondo particionamento consistente e alta disponibilidade.

O Encontro de 2009 e a Nova Era

- **2009 – Johan Oskarsson:** Organizou em São Francisco um encontro sobre sistemas de armazenamento distribuído open-source.
- **Eric Evans:** Sugeriu o termo NoSQL para batizar a conferência, redefinido como **Not Only SQL** (Sadalage & Fowler, 2014).
- **Marco:** Nascimento da comunidade moderna de bancos não-relacionais.

Gargalos do Relacional

Escala Vertical vs. Escala Horizontal: Análise

Escala Vertical (Scale-Up – RDBMS)

- **Abordagem Clássica:** Adicionar mais CPUs, memória RAM e SSDs mais rápidos ao mesmo servidor.
- **Custo Exponencial:** Duplicar a capacidade pode multiplicar o preço do hardware por 10 vezes.
- **Teto Físico:** Limite de barramento da placa-mãe e arquitetura de processador.
- **Ponto Único de Falha (SPOF):** Queda da máquina derruba toda a aplicação.

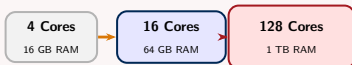
Escala Horizontal (Scale-Out – NoSQL)

- **Abordagem Nativa:** Adicionar múltiplos servidores comuns em cluster (commodity hardware).
- **Custo Linear:** Expansão elástica e gradual conforme a demanda real de tráfego.
- **Alta Resiliência:** Tolerância nativa a falhas com réplicas automáticas de dados.
- **Expansão Ilimitada:** Cluster cresce de 3 para centenas de nós sem interrupções.

Topologia: Servidor Único vs. Cluster Distribuído

Escala Vertical (Scale-Up)

Abordagem Tradicional: Servidor Centralizado

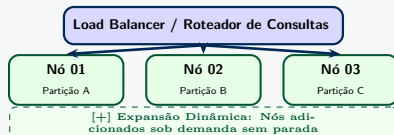


[!] Limite Físico: Teto de Barramento PCIe / Memória

- **Custo:** Exponencial (\$\$\$\$\$) a cada salto de hardware
- **Risco:** Ponto Único de Falha (*SPOF* centralizado)
- **Manutenção:** Exige paradas programadas (*downtime*)

Escala Horizontal (Scale-Out)

Abordagem NoSQL: Cluster Distribuído



- **Custo:** Linear (\$) utilizando *commodity hardware*
- **Resiliência:** Alta tolerância a falhas via replicação
- **Capacidade:** Expansão praticamente ilimitada

Takeaway: Enquanto o Scale-Up encontra barreiras físicas e financeiras, o Scale-Out viabiliza crescimento elástico.

O Gargalo dos JOINS em Clusters

Execução Local em Servidor Único

- O motor SQL resolve o `JOIN` buscando ponteiros diretos em memória RAM ou bloco de SSD local.
- Operação extremamente rápida com latência inferior a 1 milissegundo.
- O algoritmo de junção se beneficia de índices em memória e do cache local de páginas (*buffer pool*).

O Fenômeno do Network Shuffle

- Em um cluster particionado, as tabelas residem em servidores físicos distintos da rede.
- Realizar o cruzamento obriga os nós a serializar e transmitir milhões de tuplas pelos cabos e switches.
- **Impacto Severo:** O tráfego massivo satura a rede e faz a latência saltar de milissegundos para dezenas de segundos!

Anatomia do Network Shuffle: Saturação de Rede



Takeaway: Cruzar dados entre servidores físicos distintos transforma a rede no gargalo principal do sistema.

Descasamento de Impedância Objeto-Relacional

O Paradigma da Aplicação (OOP)

- A linguagem (Python, Java) trabalha com grafos ricos de objetos em memória.
- Uma entidade de negócio como `Cliente` contém atributos simples, endereços aninhados e listas de pedidos.
- Os dados estão naturalmente agrupados para o consumo da lógica de negócio.

O Atrito do RDBMS & ORM

- A teoria relacional exige normalização estrita (3NF), forçando o fatiamento da entidade em 4 ou mais tabelas.
- O mapeador (ORM) adiciona sobrecarga de conversão e problemas como consultas $N + 1$.
- **Solução em Documentos:** O formato BSON/JSON do MongoDB armazena o objeto íntegro, lido em 1 único I/O atômico!

Mapeamento Estrutural: 3NF vs. Documento BSON Nativo

Objeto em Memória (OOP)

```
class Cliente {  
  id: 1,  
  nome: "Maria Clara",  
  endereco: {cidade: "Taubaté"},  
  pedidos: [  
    {id: 101, total: 4500},  
    {id: 102, total: 120}  
  ]  
}
```

Atrito do ORM

Mapeamento 1:1

1. Abordagem Relacional (RDBMS – 3NF Normalizado)

clientes		enderecos		pedidos		itens	
id	nome	id_cli	cidade	id_ped	total	id_item	qtd

[-] Escrita: Fatiamento em 4 tabelas pelo ORM ($N + 1$ queries)

[-] Leitura: Múltiplos JOINS com produto cartesiano e tuplas repetidas

2. Abordagem Não-Relacional (MongoDB BSON Nativo)

```
{"_id": 1, "nome": "Maria", "end": {...}, "pedidos": [{...}, ...]}
```

[+] Correspondência Direta 1:1: Sem camada complexa de ORM

[+] I/O Atômico: Persistência e leitura do agregado em 1 única operação de disco!

Takeaway: Bancos de documentos alinham a persistência física à estrutura de objetos da aplicação sem camadas de atrito.

O Movimento NoSQL & Modelos

O Conceito: Not Only SQL

Origem e Filosofia

- Movimento consolidado em 2009 em São Francisco.
- Não significa rejeitar o SQL, mas sim **Not Only SQL** (Não Apenas SQL).
- **Premissa Fundamental:** Nem todas as estruturas de dados se comportam como tabelas relacionais normalizadas.
- Foco em escalabilidade horizontal nativa, esquemas flexíveis (schemaless) e alta disponibilidade.

Persistência Poliglota (Fowler & Sadalage)

- Sistemas corporativos modernos utilizam múltiplos modelos de armazenamento em conjunto.
- Escolhe-se a tecnologia mais adequada para cada domínio de negócio.
- **Exemplo Prático:** PostgreSQL (financeiro) + Redis (cache de sessões) + MongoDB (catálogo) + Neo4j (detecção de fraudes).

Orientação a Agregados: A Teoria

O Conceito Formal de Agregado

- **Sadalage & Fowler (2014) / Frozza et al.:** O conceito vem do *Domain-Driven Design* (Eric Evans).
- Um agregado é uma coleção de objetos relacionados tratados como uma **unidade indivisível de dados**.
- As atualizações dentro do agregado são atômicas e consistentes por padrão, simplificando a concorrência.

Impacto na Distribuição em Cluster

- O agregado define a fronteira natural de particionamento (*sharding key*).
- Todos os dados necessários para atender a uma requisição residem no mesmo servidor físico!
- **Não-Agregados:** Bancos de Grafos e Relacionais conectam entidades arbitrariamente, priorizando relacionamentos complexos.

Taxonomia NoSQL: Agregados, Grafos e Vetores

Modelos Orientados a Agregados (*Aggregate-Oriented*)

Sadalage & Fowler (2014) – Unidade atômica indivisível de leitura e escrita

Chave-Valor

Redis, DynamoDB

chave → valor

- Menor latência RAM
- Cache & Sessões

Documentos

MongoDB, Firestore

{ "_id": 1, ... }

- Hierarquia BSON
- Esquema flexível

Colunas

Cassandra, ScyllaDB

key → cols

- Escrita massiva contínua
- Séries Temporais / IoT

Não-Agregados

Relacionamentos ricos em rede

Grafos (Graph DB)

Neo4j, Amazon Neptune

(A) – [Aresta] → (B)

- *Index-free adjacency*
- Redes sociais e fraudes

Bancos Vetoriais (Vector Databases) – Nova Fronteira para IA & LLMs

- **Principais Soluções:** ChromaDB, Pinecone, Qdrant, Milvus, pgvector
- **Princípio Fundamental:** Armazenamento de *embeddings* para busca por similaridade semântica (RAG).

Takeaway: A taxonomia divide os bancos entre os que agrupam dados para escala e os que mapeiam conexões em rede.

Modelos NoSQL: Chave-Valor & Documentos

1. Chave-Valor (Key-Value)

Exemplos: Redis, AWS DynamoDB, Memcached.

- Mapeamento direto `chave` → `valor` em memória RAM.
- Altíssima taxa de IOPS e latência sub-milissegundo.
- **Casos de Uso:** Cache acelerador, controle de sessão de login e contadores em tempo real.

2. Documentos (Document Store)

Exemplos: MongoDB, Couchbase, Google Firestore.

- Dados armazenados em documentos JSON/BSON aninhados.
- Suporte nativo a arrays, objetos embutidos e índices secundários.
- **Casos de Uso:** Catálogos de e-commerce, perfis de usuários e cadastros com esquema evolutivo.

Modelos NoSQL: Colunas & Grafos

3. Família de Colunas (Wide-Column)

Exemplos: Apache Cassandra, ScyllaDB, Google Bigtable.

- Linhas com colunas dinâmicas em anel distribuído peer-to-peer.
- Otimizado para altíssimo volume de gravação contínua.
- **Casos de Uso:** Telemetria IoT, séries temporais, métricas de monitoramento e logs analíticos.

4. Grafos (Graph Database)

Exemplos: Neo4j, Amazon Neptune, ArangoDB.

- Vértices (nós), arestas (relacionamentos tipados) e propriedades.
- Travessia direta de ponteiros sem múltiplos JOINS.
- **Casos de Uso:** Redes sociais, motores de recomendação e detecção de fraudes financeiras.

Bancos Vetoriais & IA (Modernização)

Vector Databases na Era da IA

- Dados não estruturados convertidos em vetores matemáticos multidimensionais (**embeddings**).
- Busca por **proximidade semântica** (similaridade por cosseno), não por correspondência de texto exato.
- **Exemplos:** ChromaDB, Pinecone, Qdrant, Milvus.

Aplicações Críticas: RAG

- **RAG (Retrieval-Augmented Generation):** Recuperação semântica contextual para abastecer Modelos de Linguagem (LLMs).
- **Tópico Modernizado:** Nossa disciplina contará com uma aula prática dedicada (Aula 16) sobre este modelo!

MongoDB na Prática & Conclusão

Instalação do Serviço MongoDB no Ubuntu (v8.0 / v9.0)

1. Repositório e Instalação

```
# Chave GPG oficial
curl -fsSL https://www.mongodb.org/
static/pgp/server-8.0.asc | \
sudo gpg -o /usr/share/keyrings/
mongodb-server-8.0.gpg --
dearmor

# Repositorio APT oficial
echo "deb [ arch=amd64,arm64 signed-by
=/usr/share/keyrings/mongodb-
server-8.0.gpg ] https://repo.
mongodb.org/apt/ubuntu $(
lsb_release -cs)/mongodb-org/8.0
multiverse" | sudo tee /etc/apt/
sources.list.d/mongodb-org-8.0.
list

# Atualizar e instalar
sudo apt update
sudo apt install -y mongodb-org
```

2. Gerenciamento do Serviço (systemd)

```
# Iniciar o servico mongod
sudo systemctl start mongod

# Verificar status ativo
sudo systemctl status mongod

# Habilitar inicio automatico no boot
sudo systemctl enable mongod

# Conectar via terminal interativo
mongosh
```

Porta padrão: 27017

Configuração: /etc/mongod.conf

Instalação do Serviço MongoDB no Windows

Instalação via Pacote MSI Oficial

- Baixar o instalador oficial (Community Server) no portal MongoDB.
- Selecionar o tipo de instalação: **Complete**.
- Marcar a opção "**Install MongoDB as a Service**".
- O assistente configura o serviço nativo com nome MongoDB sob a conta Network Service.
- Opcional recomendado: manter marcada a opção do **MongoDB Compass** (GUI oficial).

Comandos de Gestão e Acesso

- **PowerShell (Admin):** `Get-Service MongoDB`
- **Iniciar / Parar o Serviço:** `net start MongoDB`
ou `net stop MongoDB`
- **Painel Gráfico de Serviços:** Executar `services.msc` → MongoDB Server.
- **Adicionar ao PATH do Sistema:** `C:\Program Files\MongoDB\Server\...\bin`
- **Conexão:** Abrir o PowerShell e digitar `mongosh`.

Executando um Banco no MongoDB: Prática Hands-On

Criação & Inserção (mongosh)

Consulta & Escrita Sem JOIN

```
// Conecta ou cria o banco da aula
use lojadb;

// Insere documento agregado
db.clientes.insertOne({
  _id: 1,
  nome: "Maria Clara Silva",
  endereco: { cidade: "Taua", uf: "CE" },
  pedidos: [
    {
      id_ped: "PED-001",
      data: "2026-09-29",
      itens: [{ prod: "Notebook", qtd: 1, preco: 4500 }]
    }
  ]
});
```

```
// 1. Busca por campo embutido (dot notation)
db.clientes.find({
  "endereco.cidade": "Taua"
});

// 2. Adicionar item ao pedido
// atomicamente
db.clientes.updateOne(
  { _id: 1 },
  {
    $push: {
      "pedidos.0.itens": {
        prod: "Mouse Sem Fio", qtd: 2, preco: 85.5
      }
    }
  }
);
```

→ Operação atômica em 1 único I/O de disco!

Demonstração Prática: JSON vs 4 Tabelas Relacionais

Relacional: 4 Tabelas + 3 JOINS

```
SELECT c.nome, e.cidade, p.  
       id_pedido,  
       i.produto, i.quantidade  
FROM   clientes c  
JOIN   enderecos e ON c.id = e.  
       id_cliente  
JOIN   pedidos p ON c.id = p.  
       id_cliente  
JOIN   itens_pedido i  
       ON p.id_pedido = i.id_pedido  
WHERE  c.id = 1;
```

Exige remontar objetos a partir de múltiplas linhas tabulares com redundâncias.

NoSQL: Documento Agregado

```
{  
  "_id": 1,  
  "nome": "Maria Clara Silva",  
  "endereco": {"cidade": "Taubá", "uf": "CE"},  
  "pedidos": [  
    {  
      "id_pedido": "PED-001",  
      "itens": [  
        {"produto": "Notebook", "qtd": 1}  
      ]  
    }  
  ]  
}
```

Leitura e escrita direta em 1 único I/O atômico de disco.

Matriz Comparativa: Relacional vs NoSQL

Dimensão	Relacional (RDBMS)	Não-Relacional (NoSQL)
Estrutura	Esquema rígido e normalizado (3NF)	Esquema dinâmico e flexível (schemaless)
Escalabilidade	Predominantemente Vertical (Scale-Up)	Horizontal nativa em cluster (Scale-Out)
Consistência	Imediata e estrita (ACID)	Eventual ajustável (BASE / CAP)
Relacionamentos	Otimizado para JOINS locais	Desnormalização ou Grafos dedicados
Cenário Ideal	Sistemas bancários, ERPs, sistemas financeiros	E-commerce, telemetria IoT, Big Data, IA / RAG

Próximos Passos & Atividades

Próximo Encontro: 06/10/2026

Aula 02 – Teorema CAP e BASE

- O Teorema CAP de Eric Brewer detalhado.
- Teorema PACELC: a troca entre consistência e latência.
- Comparação formal: ACID vs BASE.

Tarefas do Estudante

- Instalar o serviço do MongoDB no Ubuntu ou Windows conforme o roteiro.
- Executar o script didático atualizado:
`codigo/demonstracao_aula01.py`.
- Iniciar os exercícios do Bloco 1 da **Lista de Exercícios 1 (AV3)**.
- Preparar o ambiente Docker para a oficina do primeiro sábado letivo (10/10).

Dúvidas & Discussão

IFCE Campus Tauá – ADS16

reginaldo.fernandes@ifce.edu.br